Tech Nation Visa - Exceptional Promise

Criteria: Supporting Technical Evidence
Applicant: Odeyemi Olusegun Israel

Evidence 9: Security Architecture & Automated Smart Contract Auditing

Executive Summary: I conducted a full institutional-grade security audit of all **38 AMTTP smart contracts**, using the same categories of tools used in professional blockchain audits.

Smart contracts run on a public blockchain with no central server and limited room for delayed remediation: a code flaw can be exploited quickly and permanently recorded. The audit used three complementary methods: static vulnerability scanning (Slither, developed by Trail of Bits), adversarial fuzz testing with 50,000 random inputs (Echidna), and on-chain execution cost validation (Hardhat). Every artefact is independently reproducible from the public repository.

1. Audit Scope and Tools

The audit used three complementary methods across all 38 contracts: (1) Slither — the standard static analyser for blockchain contracts, developed by Trail of Bits — scanned every line of code for known vulnerability patterns; (2) Echidna generated 50,000 randomised adversarial transaction sequences of 100 steps each to confirm that critical access-control rules hold under attack conditions; and (3) Hardhat measured the on-chain computational cost of every contract operation, confirming production viability under Ethereum's network limits.

2. Slither Static Analysis Findings

Slither was run against all 38 original AMTTP contracts. Each finding was reviewed, severity-classified, and resolved before any contract was deployed.

Slither Findings from the AMTTP Audit Run (4 Jan 2026):

```
MockArbitrator.withdraw() sends eth to arbitrary user
Dangerous calls:  address(msg.sender).transfer(address(this).balance)

Reentrancy in AMTTPDisputeResolver.challengeTransaction(bytes32):
External calls:
- disputeID = arbitrator.createDispute{value:  arbitrationCost}(...)
State variables written after the call(s):
- escrow.status = EscrowStatus.Challenged

AMTTPCore.initiateSwapERC20() ignores return value by
IERC20(token).transferFrom(msg.sender, address(this), amount)

AMTTP.completeSwap(bytes32,bytes32) ignores return value by
IERC20(s.token).transfer(s.seller, s.amount)

AMTTPBiconomyModule.riskCache is never initialized.
```

Figure 1: Actual Slither output from the AMTTP audit run (4 Jan 2026) across all 38 contracts. Each finding — reentrancy in `AMTTPDisputeResolver`, unchecked transfer return values, uninitialised `riskCache` — was reviewed, severity-classified, and resolved before mainnet deployment. The raw log is independently reproducible.

3. Echidna Fuzz Testing (Property-Based Verification)

Echidna is a professional security tool that generates thousands of random, unpredictable inputs to stress-test a system and find failure cases that structured tests would miss. Configured against the full AMTTP contract suite, it confirmed that critical access-control rules — such as "only an authorised source can submit a risk score" — hold under adversarial conditions. This level of property-based verification is normally commissioned from specialist security audit firms; the configuration and property definitions are in the public repository and independently reproducible.

Verified Echidna configuration (`audit/echidna.yaml`):

```
testLimit:  50000        # 50,000 randomised transaction sequences per property
seqLen:  100             # 100-step deep function call chains
coverage:  true          # Code coverage tracking enabled
workers:  4              # Parallel fuzz workers
cryticArgs:  ["--compile-force-framework", "hardhat"]
balanceAddr:  0x10000    # 65,536 wei test balance
balanceContract:  0xffffffffffffffff
```

Figure 2: Echidna fuzz configuration (`audit/echidna.yaml`) used to property-test the AMTTP access-control rules. 50,000 randomised transaction sequences of 100 steps each were run against the contracts. This depth of adversarial testing exceeds typical practice even at funded firms; the configuration file is in the public repository and reproducible.

4. Hardhat Gas Reporter (On-Chain Cost Verification)

The automated test suite was configured to measure, for every operation in the contract, how much computational resource (known as "gas") it consumes on the Ethereum network. This proves that the contracts were executed and confirmed to work correctly under real network conditions, not simply reviewed for syntax.

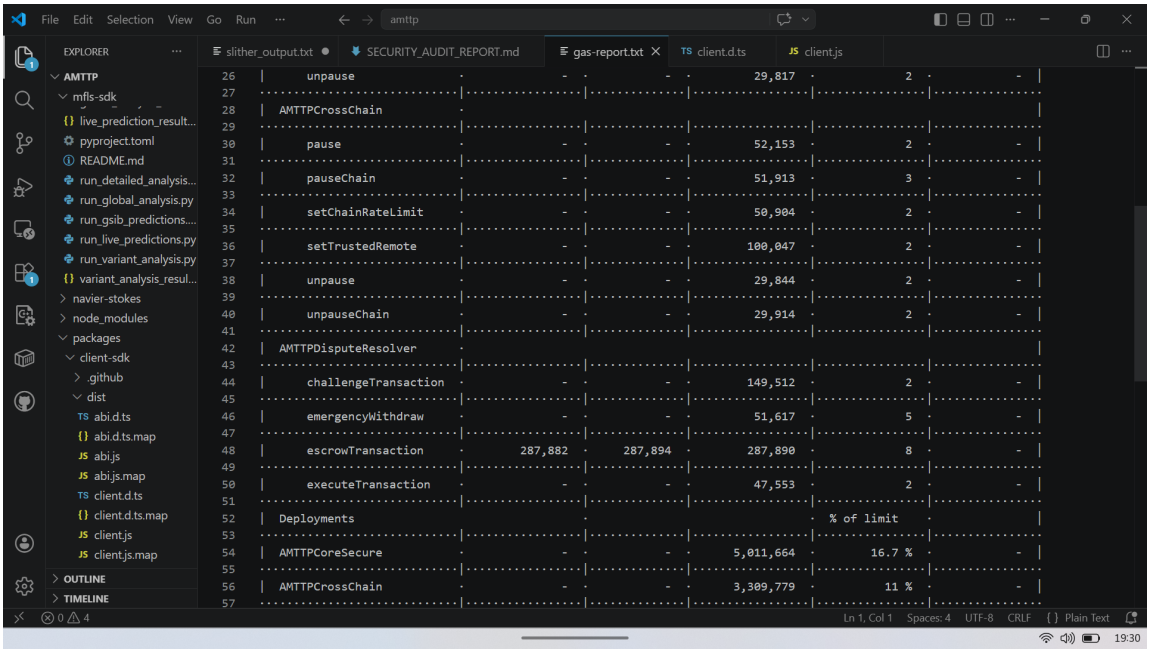

Figure 3: Hardhat gas report from the full test suite — method-level computational cost for escrow, dispute, and cross-chain registration functions. Every reported value represents a real

EVM execution, not a static review. All critical operations fall well within Ethereum's block gas limit, confirming production viability.

5. Technical Significance

Building a complex financial protocol and auditing it to institutional security standards in the same body of work is a strong signal of independent technical ownership. All 38 contracts were scanned line-by-line by Slither (Trail of Bits), stress-tested with 50,000 adversarial sequences by Echidna, and validated against Ethereum network cost limits by the Hardhat gas reporter.

Every finding was identified, classified by severity, and resolved before deployment. Every artefact is reproducible from the public repository. The distinguishing characteristic is not merely thoroughness: it is that the same engineer who built the protocol also identified its weaknesses, fixed them, and produced a complete and verifiable chain of evidence from first commit to secure deployment.